

Step 2 — Project 2's Agent, Rebuilt as a Graph With Saved State — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangGraph), **Advanced** (what actually proves the pause/resume cycle works, not just that it compiles). Read Basic first even if you already know LangGraph — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Everything Project 2's Worker loop already does — ask the model, check for tool calls, run them, ask again — becomes 2 nodes and a conditional edge instead of a `while True:` loop: an "agent" node that calls the model, and a "tools" node that runs whatever the model asked for. The conditional edge, checked after the agent node runs, decides "go to tools" or "go to end" based on whether `response.tool_calls` is empty — the exact same check Project 2's loop already makes, just expressed as graph routing instead of an `if`.

Nothing about the *behavior* changes here — same tools, same model, same decisions. What's new is that the "loop" (agent → tools → agent → tools → ...) is now a cycle in the graph, made visible, and a checkpointer can save its state after every single node, not just when the whole thing finishes.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The pieces to build:

```
# state.py
class GraphState(TypedDict):
    task: str
    messages: list # grows the Step 1 state – the conversation/tool history, not

# nodes.py
def call_model(state: GraphState) -> dict:
def call_tools(state: GraphState) -> dict:

def should_continue(state: GraphState) -> str: # the routing function
```

`call_model` invokes the tool-bound model on `state["messages"]` and returns `{"messages": [response]}` — a *list* to append, not the whole history rebuilt (LangGraph's `add_messages` reducer, or your own list-append logic, handles merging it in). `call_tools` runs whatever's in the last message's `tool_calls`, same as Project 2's Worker, and returns the resulting `ToolMessage`s the same way.

`should_continue` reads the last message: if it has `tool_calls`, return `"tools"`; otherwise return `"end"`. Wire it with `graph.add_conditional_edges("agent", should_continue, {"tools": "tools", "end": END})`, and `graph.add_edge("tools", "agent")` closes the loop back around.

For the checkpointer: `from langgraph.checkpoint.memory import MemorySaver`, `graph = builder.compile(checkpointer=MemorySaver())`, and every `.invoke()` call needs a `config={"configurable": {"thread_id": "..."}}` — the thread ID is what ties a saved run to a later resume.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A pause/resume cycle that only works because the Python process never actually stopped isn't proving what it looks like it's proving — `MemorySaver()` genuinely does keep state in memory, so a bug where you'd built a checkpointer that silently does nothing can still appear to "work" if you invoke and resume within the same running script, using the same in-memory graph object. The real test the README asks for — "prove a pause/resume cycle works... before adding any more agents" — only means something if the resume happens as a *believably separate* event: a second, independent call, ideally in a second process (`python resume.py` run right after `python main.py` finishes), using nothing but the `thread_id` to reconnect.

The second thing worth getting right at this size, before Step 3 adds a Retriever that also needs to read/write `messages`: keep `call_model` and `call_tools` reading and writing *only* `state["messages"]`, never reaching past it to also touch `state["task"]` after the first node. `task` already did its one job (seeding the first message) — a later node quietly reading it again to "double check" invites 2 sources of truth for what's actually being worked on once state has 3+ fields, which is Step 3's exact starting shape.

The extra pieces: - A second script (or a clearly separated second `if __name__` block / cell) that only imports `build_graph()`, reconnects with the *same* `thread_id`, and calls `.invoke(None, config=...)` to resume — proving the state truly persisted, not just that the object reference did. - A short comment or docstring on `GraphState` noting `task` is read-once, by the first node only.

Sketch what your "second, separate resume" test actually proves versus what an in-process resume proves, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get the loop rebuilt as a graph with a checkpointer attached — which compiles and runs. Advanced is about the difference between a checkpointer that *compiles* and one that's actually been proven to persist state outside the running program, which is the entire reason Project 3 reaches for LangGraph over Project 2's plain loop in the first place.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state: task, messages (list)

call_model(state):
    response = model_with_tools.invoke(state["messages"])
    return {"messages": [response]}

call_tools(state):
    run every tool call in the last message, same as Project 2
    return {"messages": [the resulting ToolMessages]}

should_continue(state):
    if last message has tool_calls: return "tools"
    else: return "end"

build_graph():
    add call_model as "agent", call_tools as "tools"
    entry -> agent
    agent -> (conditional: should_continue) -> tools or end
    tools -> agent
    compile with a checkpoint
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
state.py:
    from typing import TypedDict, Annotated
    from langgraph.graph.message import add_messages

    class GraphState(TypedDict):
        task: str
        messages: Annotated[list, add_messages]

nodes.py:
    from tools import calculator, lookup_weather, flaky_lookup
    TOOLS = [calculator, lookup_weather, flaky_lookup]
    TOOLS_BY_NAME = {t.name: t for t in TOOLS}
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools(TOOLS)

    def call_model(state):
        response = model.invoke(state["messages"])
        return {"messages": [response]}

    def call_tools(state):
        last = state["messages"][-1]
        results = []
        for call in last.tool_calls:
            tool_fn = TOOLS_BY_NAME[call["name"]]
            try:
                result = tool_fn.invoke(call["args"])
            except Exception as e:
                result = f"Error: {e}"
            results.append(ToolMessage(content=result, tool_call_id=call["id"]))
        return {"messages": results}

    def should_continue(state):
        last = state["messages"][-1]
        return "tools" if last.tool_calls else "end"

graph.py:
    from langgraph.graph import StateGraph, START, END
    from langgraph.checkpoint.memory import MemorySaver

    def build_graph():
        builder = StateGraph(GraphState)
```

```

        builder.add_node("agent", call_model)
        builder.add_node("tools", call_tools)
        builder.add_edge(START, "agent")
        builder.add_conditional_edges("agent", should_continue, {"tools": "tools"})
        builder.add_edge("tools", "agent")
        return builder.compile(checkpointer=MemorySaver())

main.py:
    graph = build_graph()
    config = {"configurable": {"thread_id": "run-1"}}
    result = graph.invoke({"task": "...", "messages": [SystemMessage(...), HumanMessage(...)]})
    print(result["messages"][-1].content)

```

Write the full typed version yourself, then confirm you can call `graph.invoke(None, config=config)` a second time with the same `thread_id` and get a coherent continuation (or the same finished state, if the run already completed), before moving to the Advanced version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
main.py: builds the graph, invokes it once with thread_id="verify-persistence", e

resume.py: separate script, run second (a fresh `python resume.py` process):
    graph = build_graph()    # a brand-new CompiledGraph object, same checkpointer
    config = {"configurable": {"thread_id": "verify-persistence"}}
    state = graph.get_state(config)
    print(state.values["messages"])    # proves the prior run's messages are really
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# main.py
from graph import build_graph
from langchain_core.messages import SystemMessage, HumanMessage

graph = build_graph()
config = {"configurable": {"thread_id": "verify-persistence"}}
graph.invoke(
    {"task": "What is 47 * 6?", "messages": [SystemMessage("Use the right tool.")],
    config=config,
)
print("main.py done — now run resume.py as a separate process")
```

```
# resume.py
from graph import build_graph

graph = build_graph()
config = {"configurable": {"thread_id": "verify-persistence"}}
# your turn: call graph.get_state(config) and print state.values["messages"] —
# if this prints the real conversation from main.py's run, the checkpointer genuinely
...
```

Fill in `resume.py`, run `main.py` then `resume.py` as two truly separate `python` invocations, and confirm the messages really carried over — then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: the graph itself (2 nodes, one conditional edge, a cycle, a checkpointer) is identical at all 3 levels. Advanced is entirely about the

test: proving persistence across a real process boundary, the only kind of test that actually rules out "it only looked like it worked because the Python object never left memory."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)